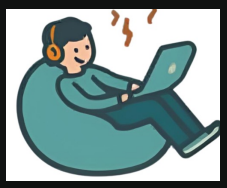

The Lazy Developer

03

MODULE

Never lose data again with backups and version control.

The module may save your life one day! Learn to setup a backup script. Build out your Github repo. Work with branches, and other strategies to protect your code from being overwritten and always give you the opportunity to roll back if needed.

Table of Contents

01. Let's talk about backups.

10 min

The “Spare Keys” Philosophy

Treat every chunk of digital work like code, photos, meeting notes, random /.txt ideas like the keys to your house. You keep a set in your pocket, another with a friend, maybe one hidden under a plant pot. Same concept here: a working copy that’s always with you, a second copy you can reach fast, and a third that’s safe if the first two disappear. Below are the handful of tools that cover almost every developer’s needs without turning your desk into a Best /Buy branch.

---

1. Your First Line of Defence: Local Backups

Portable drives (USB C or Thunderbolt SSDs) are still the cheapest, fastest way to clone your laptop. Plug one in, let macOS /Time /Machine or Windows /File /History run nightly, and you’re protected from accidental deletes or that moment when your laptop decides coffee is conductive. Or, use it as your storage drive so that you can take it with you between machines.

Need something always on? A small NAS box like Synology sits on your network and quietly captures hourly snapshots of every machine. It’s the premium “never plug in again” option and the first place you’ll reach when you just overwrote main.css at 2 /a.m. Sure, you can just use the "Restore Checkpoint" in Cursor itself but this doesn't work as a strategy when you've changed 14 other files at the same time. You are likely to break soemthing else particularly if this file is a server-related function rather than just aa simple style sheets file.

> *Takeaway: Keep one recent copy near you at all times for instant recoveries. Keep a second one also nearby in case that first one fails (and it will fail at some point!)*

---

2. The Safety Deposit Box: Cloud Storage

Even the best external drive can’t survive a house fire, power surge, or just stupidity of being dropped on the floor and smashing to pieces. That’s where off site copies come in:

- iCloud /Drive can mirror your entire Documents/Desktop folders. Close your MacBook, open the Desktop, same files appear.
- OneDrive, Dropbox, Box, Google Drive - pick whichever logo already lives in your taskbar; they all version files for at least 30 /days and let you roll back a single spreadsheet without restoring the whole account. Files are accessible anywhere there is an internet connection.

If you’re on Google /Workspace (Gmail in your own domain) you’re already paying for a chunk of Drive space so use it! The point isn’t the brand name; it’s that one good copy lives somewhere far away and professionally guarded. Leverage all those free tiers if you're on a shoestring budget! Code itself doesn't take up a lot of space, but it's other things that can like package dependencies, image/video files, databases with lots of data in them.

> *Takeaway: Off site means disaster proof. Cloud sync makes it painless.*

---

3. Don't Forget the "Little" Stuff: Notes & Docs

That random configuration setting you jotted in Apple /Notes is part of the job too. Notes, wikis, and documentation collapse just as catastrophically as code if they're only on one device. If you keep information in a notes app, this is not in your documents or code folder, this now lives somewhere else!

- Apple /Notes syncs with iCloud automatically so any Apple device you has will automatically update if you've enabled iCloud sync.
- Evernote, Notion, Obsidian and friends keep a copy on every device plus their own cloud.

Whichever app you love, double check that sync is turned on and two factor authentication is enabled. Then treat those notes as seriously as source code because next month they will hold the only up to date diagram of your API. And remember, if you delete in one of those apps, the change is synchronised everywhere! So use them as a place to put information rather than a place to edit.

---

4. Code Needs Its Own Playground: Version Control Repositories

Enter GitHub. Push your project and you instantly gain (more on Github in the rest of this module):

- A web application (free!) that doubles as an off site backup. Publishes code to testing and production environments, stores all your code.

Upcoming sections in this module will cover Github extensively.

---

5. Automate the Boring Bits: Backup Scripts

Files and repos are great, but production data lives in databases and config files too. A tiny shell or PowerShell script can:

1. Dump your PostgreSQL or MySQL database to a timestamped file.
2. Tar zip your /assets folder.
3. Push both to a private bucket or separate Git repo.

Run that script nightly with cron or Task /Scheduler and you're covered. I simply asked AI to write me one based on my app's configuration.

> *"Create a backup script I can run on a schedule that will copy all files in this project folder, copy the database safely, and create a folder that allows me to name it before you backup. Then allow a restoration feature based on my tech stack."*

I'll share my own backup script at the end of this module so you can adapt it to your stack. You'll need to update the your directory name with your appropriate directory path.

```
#!/bin/bash

# Set backup directories
BACKUP_DIR="<your directory name>"
SOURCE_DIR="<your directory name>"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

# Colors for output
GREEN='\033[0;32m'
RED='\033[0;31m'
YELLOW='\033[0;33m'
NC='\033[0m' # No Color

# Prompt for backup name
read -p "Enter a name for this backup (e.g., v0.2.3-security): " BACKUP_NAME
if [ -z "$BACKUP_NAME" ]; then
    BACKUP_NAME="unnamed"
fi
# Replace spaces with underscores
BACKUP_NAME=$(echo "$BACKUP_NAME" | tr ' ' '_')

# Create backup directories if they don't exist
mkdir -p "$BACKUP_DIR/backup_${TIMESTAMP}_${BACKUP_NAME}"

echo -e "${GREEN}Starting backup at $(date)${NC}"

# Project files backup
echo -e "${YELLOW}Backing up project files...${NC}"
rsync -av --delete \
    --exclude 'node_modules' \
    --exclude '.git' \
    --exclude 'dist' \
    --exclude '.next' \
    --exclude 'test_output' \
    "$SOURCE_DIR/" \
    "$BACKUP_DIR/backup_${TIMESTAMP}_${BACKUP_NAME}/" && \
echo -e "${GREEN}Project files backup successful${NC}" || \
echo -e "${RED}Project files backup failed${NC}"

# Extract SQL from db directory for reference
echo -e "${YELLOW}Backing up SQL schema files...${NC}"
mkdir -p "$BACKUP_DIR/backup_${TIMESTAMP}_${BACKUP_NAME}/db_schema"
if [ -d "$SOURCE_DIR/db" ]; then
    cp -R "$SOURCE_DIR/db/"*.sql "$BACKUP_DIR/backup_${TIMESTAMP}_${BACKUP_NAME}/
db_schema/"
    echo -e "${GREEN}SQL schema files backup successful${NC}" || \
    echo -e "${RED}SQL schema files backup failed${NC}"
else
    echo -e "${RED}Warning: No db directory found${NC}"
fi
```

> *Takeaway: Automation beats memory. If a human has to remember, it will fail during holidays.*

Wrapping Up

With two portable drives, one cloud folder, a synced notes app, a Git remote, and a scheduled backup script, you've ticked every box of the 3 2 1 rule without spending more than a drive's price and a few free SaaS tiers. That modest setup has rescued countless projects from coffee spills, stolen backpacks, and the occasional "who deleted the production table?" panic.

What's my personal setup?

Code runs off an external U.2 M2 4TB SSD.This is copied to a 4 TB disk array.
This is also copied to iCloud and sites in a Google Drive.
This is also stored in multiple repos in Github.

Section /2 – /The Wonderful World of /GitHub

(From “what on earth is /Git?” to pushing code like a pro)

Git in 90 /Seconds

Git is a version control system: a clever bookkeeping book that sits on your laptop and records every change you make to a folder of files. You can rewind, compare versions, or spin off an experimental copy called a branch without touching the main storyline.

But a ledger no one else can see is only half the story. GitHub turns your private ledger into a shared library hosted on their servers so the future you on a different machine can clone the project, suggest edits, and run automated tests the moment code lands. Think of Git as your local save game file; GitHub is the multiplayer server plus the scoreboard and chat room.

Core Concepts You’ll Bump Into Every Day

- Repository (repo) – the project’s folder plus its entire history.
- Remote – the copy of that repo living on GitHub’s servers.
- Commit – a single snapshot in time, complete with a message explaining why the change happened.
- Branch – a parallel timeline for new work.
- Pull Request (PR) – a request to merge your branch back into the main timeline; it’s where code review happens.
- Issue – a ticket for bugs, feature ideas, or “please update the README.”
- Fork – your personal copy of someone else’s repo; perfect for open source contributions.

Once these words feel natural, GitHub stops looking like a maze of buttons and starts feeling like collaborative magic.

A Quick Stroll Around the Interface

Open any repo on GitHub and you’ll see tabs across the top:

- Code – the files plus a commit graph that lets you time travel.
- Issues – bug tracker and to do list rolled into one.
- Pull Requests – proposals waiting for review; comments, inline suggestions, and automatic checks live here.
- Actions – GitHub’s built in CI/CD. You can run tests, linting, or even deploy to production on every push.

- Projects – a kanban board to visualise work in progress. Drag an issue from “To /Do” to “Done.”
- Wiki – project docs that anyone with access can edit—great for architecture diagrams or onboarding guides.
- Security – Dependabot alerts, secret scanning, and other “keep the bad guys out” tools.
- Insights – graphs showing commit velocity, PR throughput, and more; handy for retros.

You don’t have to master every tab on day one. Most developers live in Code, Issues, and Pull Requests; the rest become useful as the project grows.

What Does It Cost?

For solo devs and small teams, GitHub Free is surprisingly generous: unlimited public and private repos, plus 2,000 minutes of GitHub /Actions runners each month and 500 /MB of package storage. That’s enough to run unit tests on every push and still have minutes to spare.

The headline: you can get started today with zero budget and upgrade only when your workflow outgrows the free tier.

A Day in the Life: Real World Workflow (just ask Cursor to set this up for you!)

1. Clone the repo to your laptop and create a branch: git checkout -b feature/login throttle.
2. Code away; each logical chunk becomes a commit with a clear message.
3. Push the branch; GitHub opens a Pull Request automatically.
4. Actions kick off: linter, unit tests, maybe a staging deploy.
5. Teammates review, comment inline, and hit Merge when happy.
6. The main branch triggers another Action that ships a Docker image to production.

Every step is saved, audited, and crucially, recoverable. If that new throttling logic accidentally blocks all users, you can revert to the earlier commit in seconds.

Bonus, let's say you have a desktop at the office and a laptop. You go work somewhere else or on holiday and want to code? You can pull from github the latest update you pushed from your home machine before you left. Then you work on your laptop while on holiday. Push to github, when you get home, you pull down your updates. AI will handle the commands, just tell it that you had been away and working on your laptop. No need to carry around a storage drive!

Key Takeaways

1. Git keeps history; GitHub adds collaboration and automation.
2. Learn the vocabulary: repo, branch, commit, PR, issue, and you’ll navigate 90 /% of daily tasks.

3. Most essential features are free; costs scale with team size and CI minutes.
4. A simple flow of branch !’ commit !’ pull request !’ merge protects your main code line and leaves an audit trail.

With these basics under your belt, you’re ready to set up your own repository (next section) and push that very first commit into the cloud where it can’t vanish with a laptop spill. Happy coding!

Section /3 – /Setting /Up /Your /Repository

(From blank screen to “Hello, world” on GitHub in under 15 /minutes)

Step /1: Create the Repo in the Cloud

Open GitHub in your browser, click the '• dropdown in the upper right, and choose New /repository. Give it a short, memorable name (portfolio site, todo api) and decide whether the world can see it (public) or only your team (private). That’s it, now your remote copy is born.

Prefer the command line? After installing the GitHub /CLI, you can run:

```
gh repo create portfolio-site
```

The interactive prompts walk you through the same options without leaving Terminal. Get stuck? Just ask Cursor to do this step for you in Agent mode.

Step /2: Prepare Your Laptop

1. Install /Git
2. Tell Git who you are (this stamps every commit):git config --global user.name "Alex Smith"
   git config --global user.email "alex@example.com"
3. Pick a default branch name. Git versions default to main, but you can be explicit the first time you initialise a repo:git init -b main

---

Step /3: Bring the Two Worlds Together

Option /A – /Clone an empty repo you just made online

```
git clone git@github.com:alexsmith/portfolio-site.git
cd portfolio-site
```

Git gives you the project folder pre wired to GitHub.

Option /B – /Turn an existing local folder into a repo and add a remote

```
cd my_local_project
git init -b main
git remote add origin git@github.com:alexsmith/portfolio-site.git
```

Both paths end at the same place: a local repository whose origin remote points at GitHub. You can run git branch to see what branch you're now sitting in.

Step /4: Make Your First Commit

Before you start hacking, drop in two foundation files:

File	Why it matters
README.md	Home page for humans: what the project does, how to run it.
.gitignore	Tells Git to skip OS cruft, build artefacts, .env secrets. A later section in this module covers this file.

Now, run git status and you will see what files have changed and need to be committed.

Add and commit:

```
git add README.md .gitignore
git commit -m "initial project scaffolding"
```

That one liner becomes the first node in your project's timeline.

Step /5: Push, Pull, and Double Check the Wiring

Send your commit to the cloud - don't forget, you can always ask Cursor to grab the status, the branch, and then commit and push for you! You just need to approve each step.

```
git push -u origin main
```

-u ties the local main to the remote main so future git push/git pull commands know where to go.

Not sure everything points the right way? Verify:

```
git remote -v
```

You should see origin twice—once for fetch, once for push—both using the same GitHub URL.

From now on:

- git pull grabs updates others made on GitHub.
- git push sends your local commits back.

And because your config (name/email) is global, every future repository you create on this machine will already know who's making the changes.

The Road Ahead

You now have a living repository: a local clone in sync with a remote copy, history tracking who changed what and when. In the next section we'll wire up SSH keys so you can ditch password prompts forever and start branching with confidence.

For today, celebrate that first green check mark on GitHub now that your code is officially backed up, share able, and ready for the world (or just your team) to see.

Section /4 – /Connecting to GitHub over /SSH

(Your choice: a few Terminal commands or a couple of clicks in the browser.)

Why Trade Passwords for SSH Keys?

Typing a password, or worse, pasting a personal access token, every time you push code is slow and risky. SSH keys swap that chore for a silent cryptographic handshake. You gain:

- Speed – no prompts, so git push feels instant.
- Security – your private key never leaves your machine; there's nothing reusable for a phisher to steal.
- Automation friendliness – CI, backup jobs, and deploy scripts can authenticate without storing plaintext secrets.

1þ ã Generate Your Key Pair

1. Visit https://github.com/settings/keys while signed in.
2. Click "Generate new SSH key pair" (GitHub will run ssh-keygen for you in the cloud, then give you the public key to download).
3. Save the private key it provides into your ~/.ssh folder (follow the on screen instructions).

2þ ã Add the Public Key to GitHub

Website Method (most common)

1. Copy your public key.
2. On GitHub, click your avatar !' Settings !' SSH and GPG keys !' New SSH key.
3. Paste the key, name it (e.g., "MacBook /Pro"), and save.

3þ ã Let the ssh agent Remember the Key - again, just ask Cursor if you get stuck with this step!

```
eval "$(ssh-agent -s)"          # start the agent
ssh-add ~/.ssh/id_ed25519       # load the key once per login
```

> *macOS & Windows 11 tip: both OSes can auto load keys at login; GitHub Docs show the toggle paths.*

4þ ã Test the Connection

```
ssh -T git@github.com
```

You should see:

```
Hi "name"! You've successfully authenticated, but GitHub does not provide shell
access.
```

If you get “Permission denied (publickey)” double check:

- The .pub key is saved in Settings !’ SSH and GPG keys.
- ssh-add -l lists your key in the agent.

5þ ã Keep That Private Key Private

Good habit	Why it matters
Use a passphrase	A stolen laptop "` stolen repos.
Back up encrypted	Losing the key can strand automated deploys.
Rotate keys yearly	Minimises blast radius if one leaks.
Hardware tokens (YubiKey, Touch ID)	Store the key in silicon; requires a tap or fingerprint per push.
Cull old keys	Remove any device you no longer own from GitHub settings.

- Section /5 – /Understanding /Branches

(Think of branches as “choose your own adventure” paths for your code.)

---

1. /Meet the main branch – the source of truth

When GitHub spins up a new repository it creates one default branch, now called main. Whatever sits on main is the public facing, “this actually works” version of your app. Most deployment tools like Netlify, Vercel, Heroku, fly.io, etc watch main and redeploy every time it changes.

Why you shouldn’t code directly on main:

- If today’s experimental commit breaks the login page, everyone pulling the repo gets a broken build.
- Rollbacks are harder; you have to surgically remove bad commits instead of just abandoning a branch.

Think of main as the production kitchen. You cook new dishes in the test kitchen first, then bring them out to diners.

---

2. /Spin off paths: feature and fix branches

Instead of editing the live novel, you make a photocopy, scribble changes, then merge back when the chapter is polished. That photocopy is a branch.

```
git checkout -b feat/user-dark-mode
```

Now when you run git branch you will see in the terminal window what branch you're in. This is also shown on the bottom left of the Cursor window.

Work, commit, push. GitHub shows it as a parallel timeline that doesn’t touch main until you say so. The benefits:

- Isolation: your half baked dark mode CSS can’t break your co worker’s hot fix.
- Easy preview: connect your hosting platform to create a “preview site” from every branch. Product managers can click a unique URL, test the feature, and leave comments—without ever touching production.
- Safe experiments: if the idea flops, delete the branch. Main is still pristine.

For longer projects teams often create steady, long lived branches like dev or staging. Developers merge small features into dev all week; a QA server auto deploys that branch so testers always see the latest build. Once dev looks good, you promote it to main with a single pull request. What you're building is a scaled down version of this as a solo developer.

3. /Naming your branches so humans understand them

A few popular patterns:

- feat/ for new functionality – feat/payment-webhooks
- bugfix/ or fix/ – fix/missing-meta-tags
- chore/ for non user facing tasks – chore/update-dependencies

Clear names answer two questions at a glance: what are you doing? and why does it exist?

---

4. /How the merge actually happens

When your branch is ready, open a Pull Request on GitHub. This is a conversation space where teammates review the diff, leave comments, and trigger automated checks (tests, linting, accessibility scans). Github's website will automatically detect new pushes and offer a pull request for you.

Behind the scenes Git offers three ways to combine timelines:

Strategy	What you see in history	When it's handy
Fast forward	Git just moves the main pointer forward (no extra commit)	Small, linear feature branches
Merge commit	Adds one "Merge branch /…" node tying the two histories	Keeps entire branch history intact; default on GitHub
Rebase	Rewrites your commits so they look like they were built on top of the latest main	Produces a clean, straight line; great for tidy history but requires care

Most teams accept the default merge commit workflow. If the history starts to look spaghetti like, you can squash or rebase before merging.

---

5. /What about conflicts?

Conflicts happen when two people edit the same line in the same file on different branches. Git stashes both versions and asks you to decide which one wins. The file will show markers like:

```
<<<<<<< HEAD
old code
=======
new code
>>>>>>> feat/user-dark-mode
```

Delete the markers, keep the correct content, commit the file, and finish the merge. If you ever get confused, copy and paste the conflict code and share it in you chat window and let Cursor help you modify it.

6. /Two developers, one repo – a day in the life - let's say a small team is working on an app.

1. Alice creates feat/upload-avatar. Bob creates fix/email-typo.
2. Each pushes to their own branch; GitHub spins up two separate preview links.
3. Reviewers approve Bob's tiny fix first. He merges—main redeploys on whatever server platform you're using as it is synced to the main branch.
4. Alice pulls main into her branch (git pull origin main) to absorb Bob's change, resolves one minor conflict, then merges. If she didn't pull main, she would not see Bob's fix.
5. Main redeploys again: now both the fix and the avatar upload are live, and nobody accidentally overwrote the other.

Key Takeaways

- Main = production. Keep it stable for everyone's sanity.
- Branches are cheap. Create one for every feature or fix; delete when done.
- Preview/staging branches let you test in the wild before customers see anything. (more on this in another section in this module)
- Pull requests provide review, automated tests, and a tidy audit trail.
- Clear names + small scope minimise merge conflicts and make history readable.

Master this branch mindset and collaborating, even across time zones, becomes smooth, predictable, and virtually panic free.

Section /6 – /Files to Ignore with /.gitignore

(Keep your repo lean, secure, and drama free.)

Why bother ignoring files?

Every time you /git add /. you're offering Git everything in the folder. That's handy until you accidentally commit:

- Secrets – .env files lighting up with API keys.
- Gigantic folders – node_modules/ alone can be hundreds of megabytes.
- Noise – OS junk (.DS_Store) and IDE metadata that change constantly and litter your history.

A good .gitignore acts like the nightclub bouncer: only the code and docs that should be version controlled get through the door.

The anatomy of a solid .gitignore

Below is a walk through of a typical full stack JavaScript project's ignore list. Here's a copy of my .gitignore file.

```
##############################
# 1. Node dependencies & logs
##############################
node_modules/          # save everyone's bandwidth
npm-debug.log*         # npm crash logs
yarn-error.log         # yarn crash logs
pnpm-debug.log         # pnpm crash logs
*.log                  # catch-all for stray log files

##############################
# 2. Environment / secrets
##############################
.env
.env.*                 # .env.local, .env.staging, etc.
!.env.example          # template stays so newcomers know the keys to set
.npmrc                 # can contain private registry tokens

##############################
# 3. Build outputs
##############################
dist/
build/
out/
.next/                 # Next.js
```

Feel free to prune anything you don’t use; the point is to catch risky or bulky files before they ever reach the remote.

One file to rule them all: ~/.gitignore_global

Some ignores belong everywhere, think .DS_Store, Thumbs.db, or your personal .vscode/. Instead of copy pasting them into every project, create a global file (ask Cursor to help you do this!):

```
touch ~/.gitignore_global
git config --global core.excludesfile ~/.gitignore_global
```

Add universal junk once and forget about it; each repo’s local .gitignore can focus purely on project specific rules.

Key takeaways

- Security first: never commit secrets or tokens—ignore or encrypt them.
- Lightweight repos clone faster and merge cleaner. Keep build artefacts and dependencies out.
- Use the tools: GitHub templates and gitignore.io cover 90 /% of cases.
- Global ignore keeps personal clutter out of every repo.

A few minutes spent perfecting your .gitignore today will save hours of “oops, I pushed my 500 /MB node_modules” tomorrow and keep your secrets out of public view.

Section /7 – /Private /or /Public? Choosing the Right Visibility for Your Repo

Imagine your GitHub repository as either a locked office (private) or a glass walled workshop on a busy street (public). Both have value but you just need to know what you're building and who should see it.

---

1 / /I / /What the Visibility Switch Actually Does

Setting	Who can see & clone	Typical use cases
Private	Only owners + explicitly invited collaborators	Proprietary code, unreleased products, early experiments, client work under NDA
Public	Anyone on the internet (read only by default)	Open source libraries, portfolio pieces, community docs, "show your work" learning projects

You can flip between the two at any time in Settings /!' /Danger /Zone /!' /Change /Visibility. Turning public !' private immediately locks the door for everyone except current collaborators; going private !' public opens the floodgates so double check no secrets or licensed assets slipped in first.

---

2 / /I / /Why Go Public?

1. Community Power—Popular projects like React (MIT license), TensorFlow (Apache /2.0), or FreeCodeCamp thrive on outside bug reports, pull requests, and discussion.
2. Portfolio & Hiring—Recruiters love a code sample they can browse in seconds. A well written README and clean commit history often beat a PDF résumé.
3. Shared Learning—Open sourcing your handy log parser or UI component lets thousands avoid reinventing the wheel. That good karma usually comes back as stars, followers, or even future job leads.
4. Licensing Freedom—By slapping on a permissive license (MIT/Apache /2) you tell the world exactly how they may use your work therefore no awkward emails are required.

> *Pay it forward tip: If your product relies on open source tools, consider releasing adjacent helpers (CLI scripts, docs generators, tiny libraries) back to the community. You benefit from bug fixes and goodwill, and they get a useful building block.*

---

3 / /I / /Reasons to Stay Private (for Now)

- Trade secrets—Core business logic, proprietary algorithms, or anything that gives you a competitive edge.
- Early drafts—Messy experiments where half the code is TODO comments; no need for public eyes yet.

- Client work—Your contract probably forbids exposing the code.
- Security reviews—Keep the repo private until you've scrubbed secrets and passed an internal audit.

Remember, you can always start private and flip to public later once the codebase is clean and licensed.

---

4 / /l / /Mix and Match: Hybrid Strategies

Approach	How it works	Example
Open core	Core library public; enterprise plugins private	GitLab's free CE vs. paid EE features
Dual license	Same code, different terms (e.g., GPL for OSS users, commercial license for SaaS vendors)	MySQL historically used this model
Sponsorware	Code stays private until sponsorship goal met, then opens up	Caleb /Porzio's Laravel Livewire approach raised ~$100 /k/yr via GitHub Sponsors

5 / /l / /Monetising a Public Repo

1. GitHub Sponsors—Add a Sponsor button; supporters pledge monthly amounts. GitHub has channelled $40 /M+ to maintainers so far.
2. Paid support/consulting—Offer priority issue triage, feature development, or training.
3. Hosted SaaS—Open source the code but charge for an easy button hosted version (the WordPress model).
4. Merch & courses—T shirts, docs bundles, or premium video tutorials tied to the project.

Open source isn't automatically a charity, it's a distribution and collaboration model. Revenue streams keep maintainers' lights on.

---

6 / /l / /What Happens When You Flip the Switch?

- Clones people already have will keep working. They just lose the ability to pull future updates if the repo goes private.
- Forks of a repo that turns private remain, but they become disconnected; upstream PRs won't work.
- URLs & stars remain intact when going public !' private !' public again, but any secret data accidentally committed might already be cached or indexed so scrub before you open.

Make a checklist: remove .env files, purge large media, confirm license headers, then change visibility.

Key Takeaways

1. Private for secrets, client work, or messy drafts; public for collaboration, portfolio, and community impact.
2. Public doesn't equal "free for all"—attach a license and set clear contribution guidelines.
3. You can convert later, but scrubbing sensitive data after it's public is nearly impossible.
4. Open sourcing even small utilities is a great way to pay it forward and sometimes even pay the bills.

Choose the visibility that fits today's goal, document your decision, and know that GitHub's switch is always there if your needs change tomorrow.

Section /8 – /Syncing /GitHub with Your Deployed /Web App

(We'll focus on Vercel as a starting point.)

---

1 / /I / /Plugging Vercel into Your Repo (one time setup)

1. Import project – In the Vercel dashboard, click New /Project !' Import Git Repository.
2. Pick the repo – Vercel asks GitHub (you will authenticate to your own Github through Vercel) for a list, you choose your app, and hit Import.
3. Build settings – Vercel detects frameworks automatically (Next /.js, Astro, etc.). Change the build command or output folder only if you have a custom setup. (Tip: take a screenshot of this and ask Cursor what the settings should be)
4. Environment variables – Add your .env values here; they're encrypted and available at build time. (Remember: these are you client variables, not server variables!)

From that moment on, every push to any branch triggers a preview deployment.

---

2 / /I / /How Branches Map to Environments

Git Branch	Vercel Environment	URL Example	Who Should See It
main	Production	myapp.com	Your real users
feat/login-dark-mode or any PR	Preview	login-dark-mode.myapp.vercel.app	Team, QA, and your clients as this is public!

- Preview deployments spin up automatically when you push to a non main branch or open a pull request. They're isolated copies which are perfect for demos, or sending to your clients to see how it looks.
- When the PR is merged into main, Vercel builds again and promotes that output to Production.

> *Good habit: never click "Promote to Production" on a Preview build in the dashboard. If you bypass the pull request merge, GitHub's history (and your audit trail) drifts out of sync with what's live. Always land changes by merging into main, even if it's your own project. ([Vercel][3])*

---

3 / /I / /Branch Protection Keeps Main Safe

In GitHub settings, add optional ules like:

- Require 1 /review before merging.
- Block merges if checks fail (unit tests, linting).

These guard rails mean nobody, yourself included, can accidentally break production with an unchecked push.

---

4 / /I / /Automated Deploys: Native Hooks vs. GitHub /Actions

- Vercel hook – The default integration already listens for push and pull_request events; nothing to configure.
- GitHub /Actions – Use if you need extra steps (e.g., database migrations). Write a workflow that runs tests, then calls vercel deploy --prod only after everything passes. This is a bit more advanced, but may be worth chatting with Cursor about if you think it's useful for your app.

Railway follows the same rhythm: connect your repo, choose which branch auto deploys, and optionally extend with GitHub /Actions for preview database branches or custom build steps.

---

5 / /I / /Rolling Back—Your Safety Net

Vercel

1. In the dashboard, open Deployments !’ Production.
2. Click the ••• menu next to a previous green build and choose Promote to Production. Vercel swaps the DNS so traffic points to that commit meaning instant rollback.

See the various deployments in my Vercel project above. You can see under each random name it shows "Production" or "Preivew". In the middle you can also see what branch they're connected to.

---

6 / /I / /Two Person Workflow Example (Vercel)

1. Dev A pushes feat/chat-emojis !’ preview URL pops up, client checks.
2. Dev B fixes a hot bug on fix/null-avatar !’ opens PR, tests pass, reviewer approves, merges to main !’ Production rebuilds and deploys automatically.
3. Dev /A rebases onto the latest main, resolves one CSS conflict, PR gets green checks !’ merges !’ Production updates again.

Nobody touches production directly; each change was visible on its own preview site first. Then, Vercel gives you a super easy way to promote a previous build if something isn't right instantly.

Key Takeaways

1. Connect once, then code – Vercel (or Railway) auto deploys every push.
2. Preview branches let you share work in progress without endangering users.
3. Merge to main !’ Production keeps Git history, CI checks, and deployments in lock step.

4. Branch protection + automated checks = fewer “it worked on my laptop” incidents.
5. Rollbacks are just “promote an older build” or redeploy an earlier commit to be fast and painless when things go sideways.

Master this flow and shipping becomes: push !’ preview !’ PR !’ merge !’ live, with confidence at every step.

Section /9 – /Keeping Your Documentation /Up /to /Date

(Small team edition: written for the “just me” coder or a two person duo.)

Why bother writing anything down?

Think about the last time you dug up an old side project. /You probably spent half an hour asking, “Which version actually works?”, “What’s this v2 final final folder?”, and “Where did I hide the API key?” Good docs mean:

- Zero detective work when you return months later.
- A single source of truth your teammate can skim instead of Slacking you at 10 /p.m.
- Clear release notes you can paste straight into emails, client updates, or store listings.

We touched on this back in Module /2 when we showed how a tidy README gives ChatGPT instant context. The same habit keeps humans productive and impresses any future employer or customer who peeks at your repo.

---

1. Version numbers & CHANGELOG: your project’s diary

Start by dropping two tiny files in the repo root:

- VERSION – just one line: 1.0.0
- CHANGELOG.md – a running diary of “what changed and why”

Follow “Semantic Versioning”—that fancy term just means:

- PATCH (0.0.x) for quick fixes
- MINOR (0.x.0) for new, compatible features
- MAJOR (x.0.0) when you break something on purpose

When you tag a commit v1.1.0, anyone (including future you) immediately knows roughly how risky it is to upgrade.

Tagging, in plain English

Think of a tag as a sticky note you slap onto a specific commit that says, “This /one is /v1.0” or “Ship it release.”
Unlike a branch, that sticky note never moves, no matter how many commits you add later, so you can always jump back to the exact code that shipped on a given day. It’s your project’s time machine bookmark.

Why tags matter for a small (1 2 person) team

- Reliable rollbacks: if today's deploy goes sideways, git checkout v1.3.2 puts you back on safe ground.
- Clear hand offs: when you tell a collaborator "build against /v2.0", they grab that tag and know they have the same code you tested.
- Professional releases: GitHub turns tags into downloadable zip/tar balls and release pages—handy for clients or app store submissions. ([GitHub Docs][2])

Two kinds of tags

Type	How it's stored	Typical use
Annotated	Full Git object with your name, date, message (and optional GPG signature)	Official releases—recommended!
Lightweight	Just a name pointing at the commit, no extra data	Quick local bookmarks you may delete later

Git and GitHub treat annotated tags as first class citizens: for example, git describe and GitHub's Releases page ignore lightweight tags by default. ([Git SCM][3], [Git SCM][4])

Creating and pushing a tag (CLI)

```
# after your last commit for the release
git tag -a v1.4.0 -m "chore: bump version to v1.4.0 — adds dark mode"
git push origin v1.4.0      # push a single tag
# or
git push --tags             # push everything you've tagged locally
```

Now the tag appears in GitHub's Code !' Tags dropdown and can be used to draft a release page with autogenerated notes.

Doing it in the GitHub website

1. Open Releases !' Draft new release.
2. Type a new tag name (v1.4.0); GitHub will create it for you if it doesn't exist.
3. Click Generate release notes (based on merged PR titles) and publish.

This creates an annotated tag under the hood and attaches your nicely formatted changelog.

---

2. Let GitHub (or AI) write the boring bits for you

Commit messages like fix: handle payment errors or feat: dark mode toggle look tidy and let GitHub auto generate release notes:

1. After merging a feature branch, click Releases !' Draft new release.
2. Pick the new tag (v1.1.0).
3. Hit Generate release notes – GitHub lists every PR title since the last tag.

Or, even easier, just ask Cursor to write you more elaborate notes when you ask it to commit!

You get professional looking notes without copy pasting from memory.

---

3. Little badges, big confidence

At the very top of README.md, add status badges from shields.io (ask Cursor to do this for you!):

- Build – green means "the tests pass"; red means "don't pull this yet."
- Coverage (if you run tests) – tells you how much code is exercised.
- Latest release – shows the v1.1.0 you just cut.

A visitor can glance at those three icons and trust the project is alive.

---

4. Docs as code: same workflow, zero extra apps

Instead of keeping a Google Doc that drifts out of date, write docs in Markdown right beside your code.

- Tiny library? Plain README.md files are enough.
- Bigger app? Drop docs in a docs/ folder and let MkDocs (Python free) or Docusaurus (React based) turn them into a website.
- Turn on GitHub /Pages and every merge to main automatically rebuilds the docs site—no manual uploads or FTP.

Because the docs live in Git, you review them the same way you review code. No extra tooling for a two person team to learn.

Putting it all together – a one person release cycle

1. Finish feat/export-csv, commit, and open a PR.
2. PR template reminds you: add a bullet to CHANGELOG.md.
3. Merge !' GitHub Actions run tests, badge stays green.
4. Draft release v1.2.0; click Generate notes !' Publish.
5. Docs site auto deploys via GitHub /Pages; README badge now shows v1.2.0.

You spent maybe three extra minutes but gained crystal clear history, instant documentation, and a project that looks as polished as any big team open source repo.

Key Takeaways (for small teams)

- Write once, read forever – your future self is the #1 user of good docs.
- VERSION + CHANGELOG + semantic versioning give every release a clear label.
- GitHub's auto release notes & README badges do most of the grunt work for you.
- Docs stored in the repo stay in sync because they follow the exact same Git workflow.

Invest a little time up front and your two person (or solo) project will feel, to outsiders and to future you, like it's run by a seasoned, well oiled team.